

성균관대학교 - 인텔리즈

- 클라우드 기반 머신비전 관리 시스템 개발 -

2023. 12. 22.

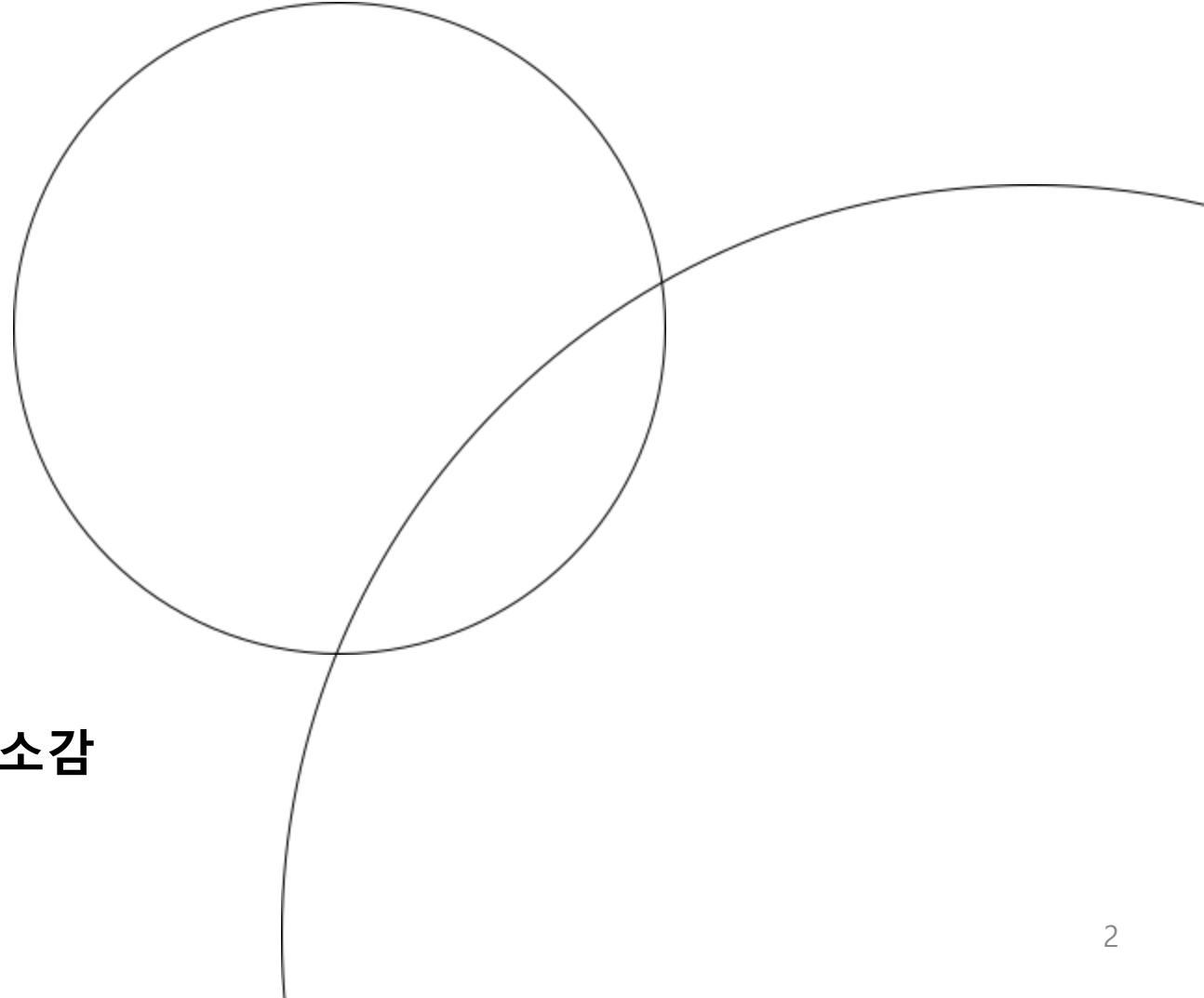
성균관대학교 소프트웨어융합대학

발표자:

이강훈	(소프트웨어학과, 2학년)
김원종	(소프트웨어학과, 3학년)
김채리	(소프트웨어학과, 2학년)

Contents

- 팀 소개
- 과제 배경
- 작품 소개
- 적용 기술
- 기술의 차별성 및 독창성
- 과제 성과 및 기대효과
- 개선점 및 향후 계획
- 시연 (시연 동영상)
- Appendix
 - 2023년 산학협력과제 수행 소감
 - 2023 산학협력 과제 요약



팀 소개



이강훈(팀장)

소프트웨어학과 / 2학년

딥러닝 평가 지표

Frontend



김채리

소프트웨어학과 / 2학년

MQTT 프로토콜

Backend



김원종

소프트웨어학과 / 3학년

DB

Backend



인텔리즈: 김선민 연구원

이원용 연구원



지도교수: 이주식 교수님

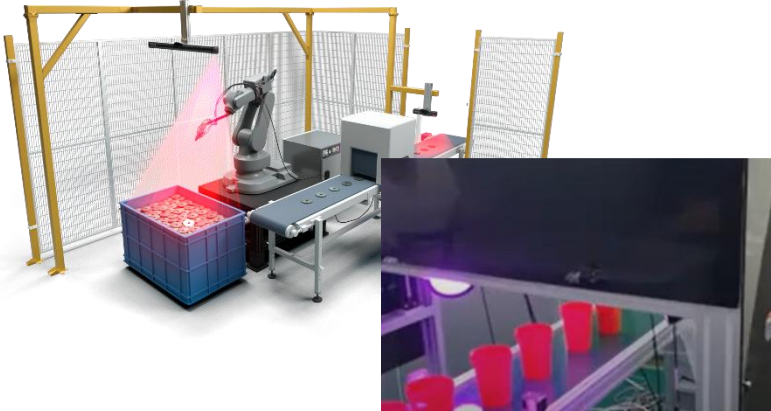
과제 배경

인텔리즈는 무슨 회사인가?

제품의 불량 여부를 판단하는 검사기의
소프트웨어와 하드웨어를 다른 기업의
제조 현장에 제공

AI 기반 머신비전 검사기를 구독할 수 있는
서비스 제공 → 각종 산업의 인건비 절약 기여

적용 사례: 플라스틱 컵의 결함 여부 판단 및 분류



문제점 및 한계

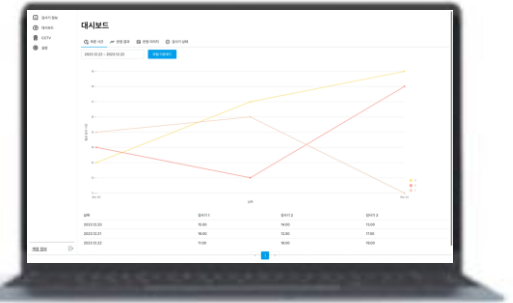
현장에 설치된 검사기 설정값이나 도출
데이터를 실시간으로 관리 및 접근하기 어려움

검사기 수정 및 문제 사항 발생 시
현장 방문 필요

즉각적 알림 기능의 부재

⇒ 고객사 확보 및 관리가 어려움

이에 대한 해결 방안: 2023 인텔리즈 산학협력팀의 과제

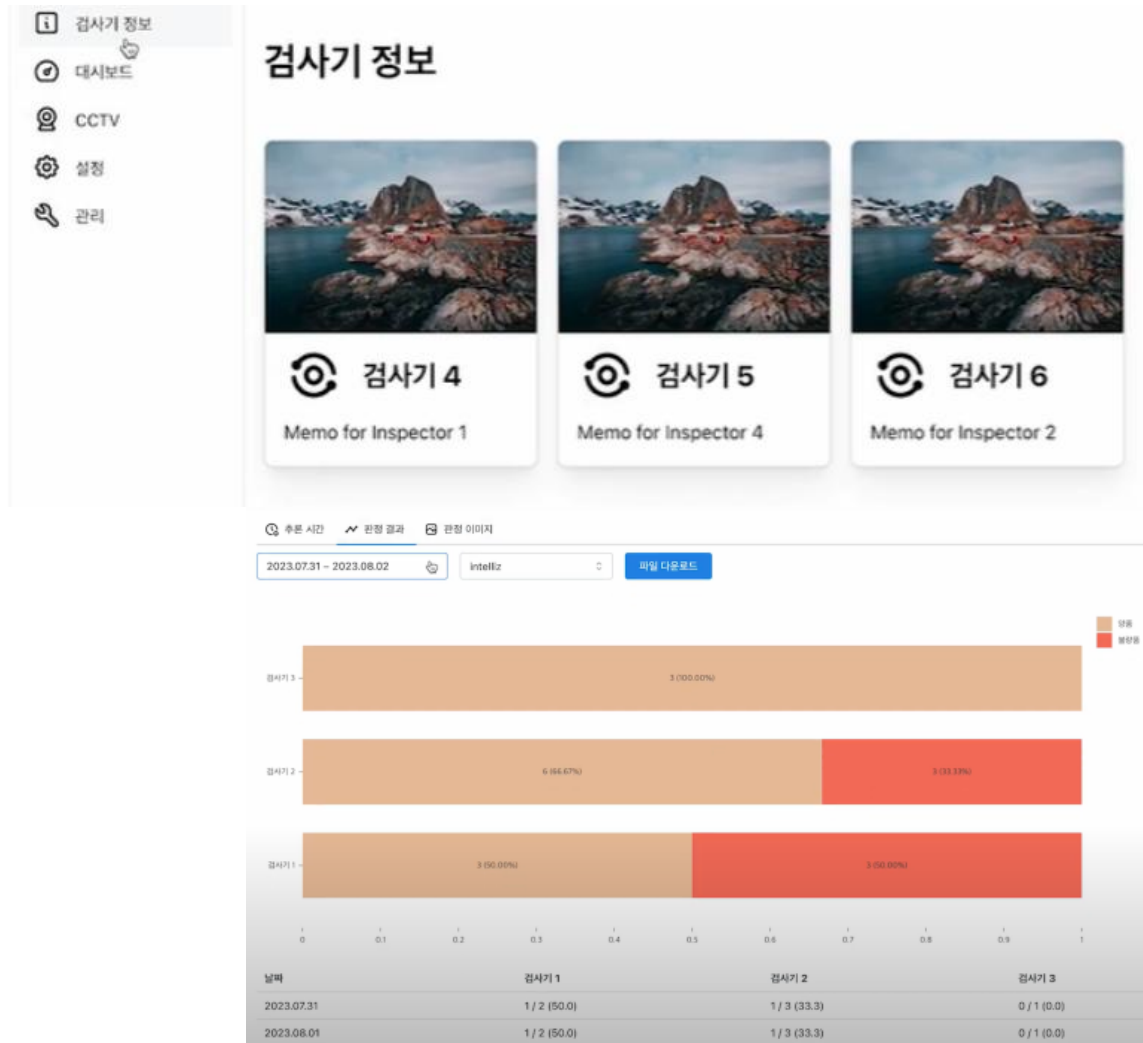


‘CAVIAR-M’ 대시보드 웹 서비스

검사기 원격 관리
검사 결과 시각화
알림 발송 서비스

⇒ 원활한 검사기 구독 서비스 운영,
부가적 서비스를 통한 경쟁력 확보

작품 소개



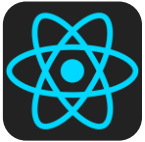
CAVIAR-M 주요 기능

- I. 실시간 검사기 설정값 및 상태
원격 조회 및 변경
- II. 검사기에서 도출된 데이터 결과값 및
시각화된 자료(불량 이미지/통계) 열람
- III. 고객사/회원 추가 및 관리
- IV. 검사기 CCTV 확인
- V. 용량 부족 및 이미지 업로드 요청 필요 시
알림 및 이메일 발송 기능

*실제 CAVIAR-M의 화면입니다.

적용 기술 - 기술 스택

Frontend



React

SPA 라이브러리,
브라우저의 객체를 동적으로
제어하기 위해 사용



Mantine

UI 컴포넌트 라이브러리,
미리 만들어진 컴포넌트로
개발 속도를 높이기 위해 사용

Backend



Django + DRF

웹 어플리케이션 프레임워크,
RestAPI 구현을 위해 함께
사용



MariaDB

오픈소스 RDBMS,
MySQL과 거의 동일한 형태의
관계형 데이터베이스

Infra



Docker + Docker Compose

가상머신에 비해 가벼운
형태의 리눅스 컨테이너 기술,
여러 컨테이너를 동시에
제어하기 위해 함께 사용



Nginx

웹 서버 / 리버스 프록시,
여러 경로를 서로 다른
컨테이너에 연결하기 위해 사용



MQTT

경량화된 발행/구독 패턴의
통신 프로토콜, 검사기 상태를
실시간으로 공유하기 위해
사용

구현 과정 - 백엔드

전체 API 및 예외 리스트

API 목록

<https://www.notion.so/6631d862dfbd4880aad990da964a0288?v=e35eb0416f8a4572aed03cb2911bc2d3&pvs=4>

☒ 완료?	URL Pattern	Method	API name	Auth	리소스
☒	v1/auth	POST	login	ALL	auth
☒	v1/auth/logout	POST	logout	CUSTOMER_NORMAL	auth
☒	v1/auth/pw_reset	POST	sendEmail	ALL	auth
☒	v1/auth/pw_reset:key	POST	resetPw	ALL	auth
☒	v1/auth/refresh	POST	tokenRefresh	CUSTOMER_NORMAL	auth
☐					
☒	v1/users	POST	createUser	ALL	user
☒	v1/users/:id	GET	getUser	CUSTOMER_SUPER	user
☒	v1/users	GET	getUsers	CUSTOMER_SUPER	user
☒	v1/users/:id	PATCH	updateUser	CUSTOMER_SUPER	user
☒	v1/users/me/register	PATCH	registerKey	DEMO	user
☒	v1/users/:id	DELETE	deleteUser	CUSTOMER_SUPER	user
☒	v1/users/me	GET	getMe	CUSTOMER_NORMAL	user
☒	v1/users/me	PATCH	updateMe	CUSTOMER_NORMAL	user
☐					
☒	v1/organizations/:orgid/inspectors	GET	getInspectors	SUPERUSER	inspector
☒	v1/organizations/myorg/inspectors	POST	createInspector	CUSTOMER_SUPER	inspector
☒	v1/organizations/:orgid/inspectors	POST	createOrgInspector	SUPERUSER	inspector
☒	v1/organizations/myorg/inspectors	GET	getOrgInspectors	CUSTOMER_NORMAL	inspector
☐					
☒	v1/inspectors/:id	GET	getInspector	CUSTOMER_NORMAL	inspector
☒	v1/inspectors/:id	PATCH	updateInspector	CUSTOMER_SUPER	inspector

예외 목록

<https://www.notion.so/15160560f4f54f659481c4bbc4b5c925?v=034d373402984500aace5a8a99033a9a&pvs=4>

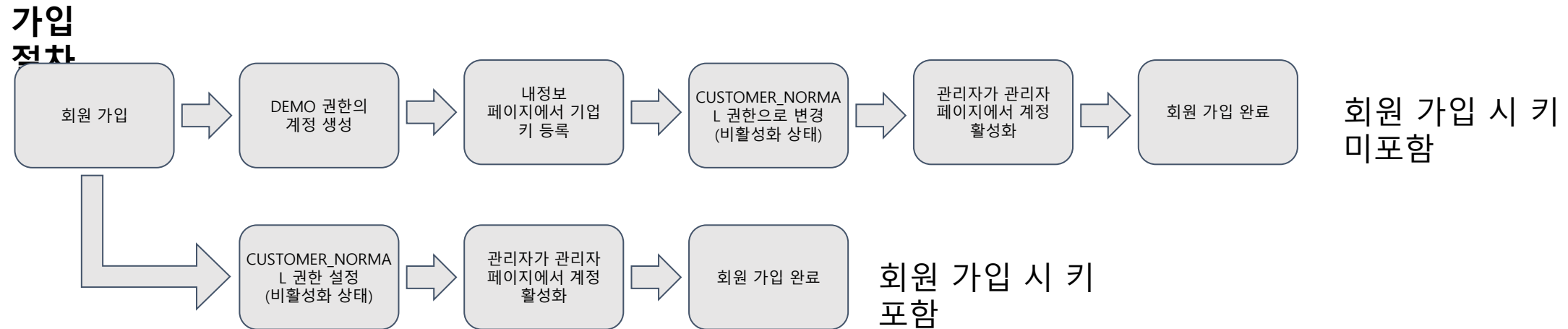
auth	로그인 불가	AccNotFound	401 Unauthorized
auth	잘못된 auth 입력값	InvalidAuthInput 1	400 Bad Request
user	잘못된 user 입력값	InvalidAccInput 1	400 Bad Request
user	허용되지 않은 user 요청	ForbiddenUserRequest OPEN	403 Forbidden
user	존재하지 않는 user 리소스	UserNotFound 1	404 Not Found
inspector	허용되지 않은 검사기 요청	ForbiddenInspectorRequest 2	403 Forbidden
inspector	존재하지 않는 검사기 리소스	InspectorNotFound	404 Not Found
inspector	잘못된 식재 주기 입력값	InvalidInspectorInput	400 Bad Request
inspectorState	허용되지 않은 검사기PC 상태 요청	ForbiddenInspectorStateRequest	403 Forbidden
inspectorState	존재하지 않는 검사기PC 상태 리소스	InspectorStateNotFound	404 Not Found
cctv	허용되지 않은 CCTV 요청	ForbiddenCCTVRequest	403 Forbidden
cctv	존재하지 않는 CCTV 리소스	CCTVNotFound	404 Not Found
organization	잘못된 고객사 입력값	InvalidOrgInput 1	400 Bad Request
organization	허용되지 않은 고객사 요청	ForbiddenOrgRequest 1	403 Forbidden
organization	존재하지 않는 고객사 리소스	OrganizationNotFound	404 Not Found

구현 과정 - 백엔드

유저 인증 - 회원 가입

키 생성 및 초기화

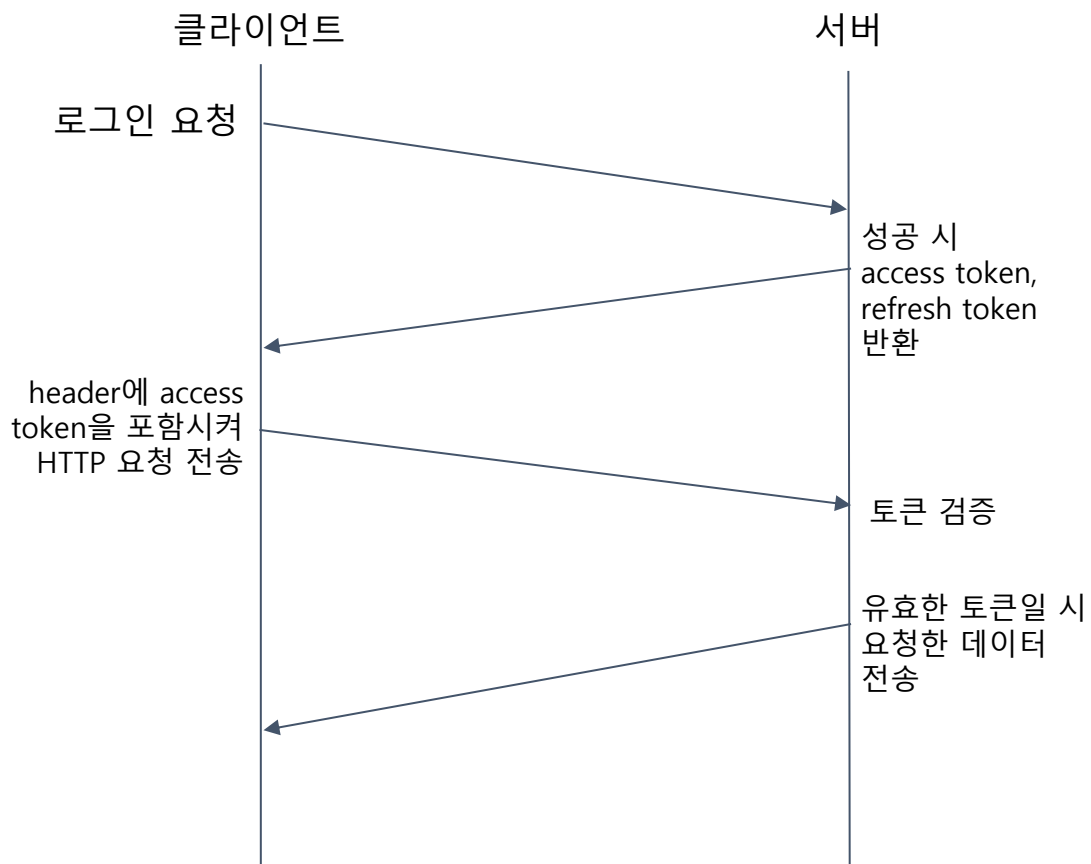
- 가입 키는 UUID4를 사용
- 중앙관리자가 고객사를 추가하면 가입 키는 자동 생성.
- 관리자 페이지에서 중앙관리자와 각 고객사 관리자 계정으로 가입 키 초기화 가능.



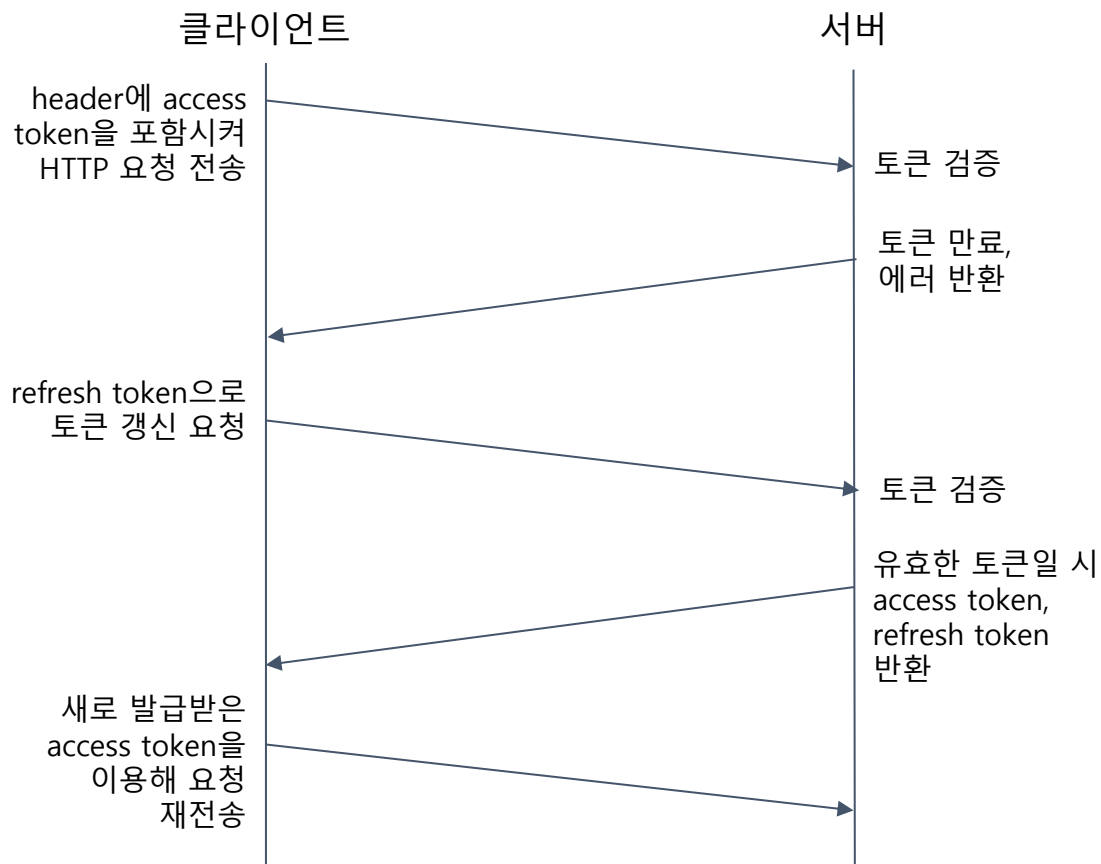
구현 과정 - 백엔드

유저 인증 - JWT 토큰

- 토큰 반환 시 access token은 response body에, refresh token은 cookie에 담아 전송
- refresh token은 1회용으로 한번 사용하면 재사용 불가
- 발급된 refresh token의 정보는 DB의 token_blacklist_outstandingtoken에 저장됨
- 사용된 refresh 토큰은 DB의 token_blacklist_blacklistedtoken 테이블에 추가되어 재사용 방지함
- 토큰 갱신 요청은 cookie에 담긴 refresh token을 사용하므로 request body가 필요하지 않음



유효한 토큰인
경우



만료된 토큰인
경우

구현 과정 - 백엔드

유저 인증 - JWT 토큰

Token 설정 변경

- settings.py 파일을 통해 토큰 옵션 변경 가능
- 기본 설정 값은 공식 문서에서 확인 가능
- [Settings — Simple JWT 5.2.2.post30+gaf92e8 documentation](#)

```
SIMPLE_JWT = {  
    "ACCESS_TOKEN_LIFETIME": timedelta(hours=2),  
    "REFRESH_TOKEN_LIFETIME": timedelta(days=5),  
    "ROTATE_REFRESH_TOKENS": True,  
    "BLACKLIST_AFTER_ROTATION": True,  
    "AUTH_HEADER_NAMES": "Authorization",  
    "UPDATE_LAST_LOGIN": True,  
}
```

토큰의 현재 설정 값

Token 인증 클래스

- token 인증 클래스는 simple jwt 라이브러리의 기본 클래스를 일부 수정해 사용 중
- backend 패키지의 authentication.py 파일에 정의함
- 해당 클래스는 settings.py 파일에서 기본 인증 클래스로 등록 됨

```
REST_FRAMEWORK = {  
    "DEFAULT_PERMISSION_CLASSES": (  
        "rest_framework.permissions.IsAuthenticated",  
    ),  
    "DEFAULT_AUTHENTICATION_CLASSES": (  
        "backend.authentication.CustomJWTAuthentication",  
    ),  
}
```

settings.py 설정

- Viewset의 permission_classes 배열에 IsAuthenticated 항목을 추가 시 settings.py에 등록된 기본 인증 클래스를 사용함.

```
5 usages chaeri +1  
class InspecDataViewSet(viewsets.ModelViewSet):  
    queryset = InspecData.objects.all()  
    serializer_class = InspecDataSerializer  
    permission_classes = [IsAuthenticated, PermissionOnInspecData]
```

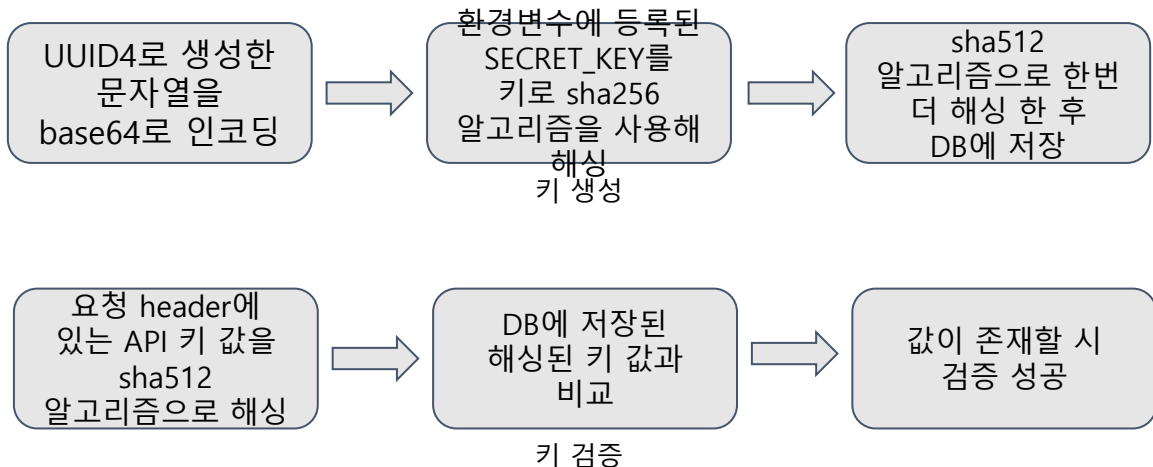
구현 과정 - 백엔드

유저 인증 - API 키

API 키 발급

- 검사 결과 이미지 업로드는 API 키를 사용해 업로드
- API 키 발급은 고객사 관리자 계정으로 발급 가능
- [generateApiKey](#) api 요청 보낼 시 응답으로 키 값이 전달 됨
- 키 생성과 검증 함수는 organization 패키지의 key.py 파일에 정의함.
- API 키는 한 번에 하나의 키만 유효함.
- 새로운 키 발급 시 기존의 키는 사용 불가

키 생성 및 검증 절차



```
def generate_key(organization: Organization) -> str:
    key = hmac.new(
        settings.SECRET_KEY.encode(),
        base64.b64encode(uuid.uuid4().hex.encode()),
        hashlib.sha256,
    )
    hashed_key = hmac.new(
        settings.SECRET_KEY.encode(), key.hexdigest().encode(), hashlib.sha512
    )
    organization.api_key = hashed_key.hexdigest()
    organization.save()
    return key.hexdigest()

1 usage  KimWonjong123
def validate_key(key: str) -> bool | Organization:
    hashed_key = hmac.new(
        settings.SECRET_KEY.encode(), key.encode(), hashlib.sha512
    )
    try:
        return Organization.objects.get(api_key=hashed_key.hexdigest())
    except Organization.DoesNotExist:
        return False
```

key.py 파일 소스코드

구현 과정 - 백엔드

유저 인증 - API 키

API 키 사용

- API 키는 검사 결과 이미지 업로드에만 사용 함
- 요청의 header에 키를 포함해서 요청을 보내야 함
- key : "API-KEY", value : "api 키 값" 형태로 넣어야 함
- 파일명 형식 등은 FileViewSet 소스코드의 주석 참조

FileViewSet의 authentication

클래스

- FileViewSet은 사진 업로드 요청에 한해 ApiAuthenticaiton 인증 클래스를 사용
- ApiAuthentication 클래스는 JWT 인증 클래스와 같이 backend 패키지의 authentication.py 파일에 정의 됨.
- 요청 종류에 따라 다른 권한, 인증 클래스를 사용하기 위해 ViewSet 내부에서 get_permissions, get_authenticators 두 함수를 오버라이드 함.

```
permission_classes_by_action = {
    "upload_file": [AllowAny],
}

authentication_classes_by_action = {
    "upload_file": [ApiAuthentication],
}

# KimWonjong123
def get_permissions(self):
    try:
        return [
            permission()
            for permission in self.permission_classes_by_action[
                self.action
            ]
        ]
    except KeyError:
        return [permission() for permission in self.permission_classes]

# KimWonjong123
def get_authenticators(self):
    try:
        return [
            authentication()
            for authentication in self.authentication_classes_by_action[
                self.action_map.get("post")
            ]
        ]
    except KeyError:
        return [
            authentication()
            for authentication in self.authentication_classes
        ]
```

FileViewSet의
소스코드

구현 과정 - 백엔드

```
class CustomJWTAuthentication(JWTAuthentication):
    """
    JWT 인증을 위한 클래스로 해당 클래스를 기본 인증 클래스로 사용함.
    settings.py의 REST_FRAMEWORK.DEFAULT_AUTHENTICATION_CLASSES에 등록되어 있음.
    token을 이용해 유저를 인증하는 과정에서 발생하는 예외를 처리하기 위해
    get_user 메서드를 오버라이딩함.
    refresh token rotation이 적용 됨.
    refresh token의 rotation 여부는 settings.py의 SIMPLE_JWT.ROTATE_REFRESH_TOKENS 옵션을 통해 설정 가능.
    refresh token 발급 시 DB의 token_blacklist_outstanding 테이블에 저장되고,
    사용 시 DB의 token_blacklist_blacklistedtoken 테이블에 저장되며 새로운 refresh token이 발급됨.
    """

    @ KimWonjong123
    def get_user(self, validated_token):
        """
        토큰의 user_id 필드를 이용해 DB에서 유저를 가져온 후 반환.
        기존의 JWTAuthentication 클래스의 get_user 메서드는 유저의 is_active 여부를
        확인해서, is_active가 False인 유저가 로그인할 수 없음.
        """
        try:
            user_id = validated_token[api_settings.USER_ID_CLAIM]
        except KeyError:
            raise InvalidToken(
                _("Token contained no recognizable user identification")
            )

        try:
            user = self.user_model.objects.get(
                **{api_settings.USER_ID_FIELD: user_id}
            )
        except self.user_model.DoesNotExist:
            raise TokenAuthenticationFailed(
                _("User not found"), code="user_not_found"
            )

        return user
```

JWT 인증 클래스

```
class ApiAuthentication(BaseAuthentication):
    """
    API 인증을 위한 클래스
    """

    @ KimWonjong123
    def authenticate(self, request):
        """
        request의 header에 API-KEY 키가 있는지 확인하고, 있다면
        organization을 반환한다.
        """
        api_key = request.META.get("HTTP_API_KEY")
        if not api_key:
            raise DRFAuthenticationFailed(
                _("Invalid api key"), code="invalid_api_key"
            )
        from organization import key

        organization = key.validate_key(api_key)
        if organization:
            return (None, organization)
        else:
            raise DRFAuthenticationFailed(
                _("Invalid api key"), code="invalid_api_key"
            )
```

API 인증 클래스

구현 과정 - 백엔드

Soft Delete

- 장고는 soft delete 기능을 지원하지 않음.
- 해당 기능을 구현하기 위해 SoftDeleteModel 클래스와 SoftDeleteModelManager 클래스를 정의함.
- 해당 클래스는 backend 패키지의 models.py 파일에 정의 됨.
- 모든 모델은 SoftDeleteModel 클래스를 상속해 구현했으며 SoftDeleteModelManager를 매니저로 사용함.
- 해당 모델의 인스턴스에 대해 delete() 함수를 호출하면 해당 인스턴스를 외래키로 참조하고 있는 모든 인스턴스의 deleted_date 필드에 삭제 시각이 기록됨.
- 예외적으로 file 모델만 soft delete 모델을 사용하지 않음.

```
class SoftDeleteModel(models.Model):
    """
    deleted_date 필드를 가지는 모델의 추상 클래스
    해당 모델의 인스턴스에 대해 delete 메시지를 호출하면
    deleted_date 필드에 삭제 시각이 기록되면
    해당 모델을 외래 키로 참조하고 있는 모델의 인스턴스들도
    함께 deleted_date 필드가 업데이트된다.
    """

    deleted_date = models.DateTimeField(null=True)

    @ KimWonjong123
    class Meta:
        abstract = True

    objects = SoftDeleteModelManager()

1 usage @ KimWonjong123
def get_referencing_models(self):
    from django.apps import apps

    referencing_models = []
    all_models = apps.get_models()

    for model in all_models:
        for field in model._meta.fields:
            if (
                field.remote_field
                and field.remote_field.model == self.__class__
            ):
                referencing_models.append(
                    {"model": model, "field": field.attname}
                )

    return referencing_models
```

```
1 usage @ KimWonjong123
def get_related_instances(self):
    referencing_models = self.get_referencing_models()

    referencing_instances = []
    for model in referencing_models:
        related_queryset = model["model"].objects.filter(
            **{model["field"]: self.pk}
        )
        for instance in related_queryset:
            referencing_instances.append(instance)

    return referencing_instances

6 usages (6 dynamic) @ KimWonjong123
def delete(self, using=None, keep_parents=False):
    related_instances = self.get_related_instances()

    if len(related_instances) > 0:
        for instance in related_instances:
            instance.delete()
        self.deleted_date = timezone.now()
        self.save()
    return self
```

```
@ KimWonjong123
class SoftDeleteModelManager(models.Manager):
    """
    SoftDeleteModel의 deleted_date가 None인 쿼리셋을 반환하는 매니저
    """

    @ KimWonjong123
    def get_queryset(self):
        return super().get_queryset().filter(deleted_date=None)
```

SoftDeleteModelManager
클래스

SoftDeleteModel 클래스

구현 과정 - 백엔드

Soft Delete - Model

- SoftDeleteModel은 model manager로 SoftDeleteModelManager 클래스를 사용
- ViewSet 클래스를 정의할 때 model 클래스의 objects.all() 함수를 호출하면 deleted_date 필드가 null인 항목만 필터링 된 쿼리셋이 반환 됨.
- Soft delete 기능은 각각의 인스턴스에 대해 delete() 함수를 호출 해야 적용 됨.
- 인스턴스가 아닌 쿼리셋에 대해 delete() 함수 호출 시 DB에서 완전히 삭제 됨.

```
def delete_organization(self, request, *args, **kwargs):
    try:
        organization = self.get_object()
    except Http404 or Organization.DoesNotExist:
        raise CustomException("0005")
    organization.delete()
    return Response({"success": "true"})
```

인스턴스에 대한 delete()

```
class InspecDataViewSet(viewsets.ModelViewSet):
    queryset = InspecData.objects.all()
    serializer_class = InspecDataSerializer
    permission_classes = [IsAuthenticated, PermissionOnInspecData]
```

InspecDataViewSet의 소스코드

```
with transaction.atomic():
    Recipe.objects.filter(inspection=inspection).delete()
    for recipe in formatted_payload.recipes:
        create_recipe(recipe)
    CCTV.objects.filter(inspection=inspection).delete()
    CCTV.objects.create(
        inspection=inspection,
        sub_id=formatted_payload.cctv.sub_id,
        video_url=formatted_payload.cctv.video_url,
```

쿼리셋에 대한 delete()

구현 과정 - 백엔드

Soft Delete -

Manager

- 추가적인 코드가 필요하지 않은 model은 SoftDeleteModelManager를 사용
- 추가적인 검증 로직이 필요한 model은 모델 매니저를 상속한 별도의 매니저를 정의해서 사용
- 현재는 Organization과 User 모델만 별도의 매니저 클래스를 사용함

```
class OrganizationManager(SoftDeleteModelManager):
    """ KimWonjong123 """
    def create(self, name):
        if not name:
            raise ValidationError(code="invalid", message="Name must be set.")
        org = self.model(name=name)
        org.save()
        return org

    """ KimWonjong123 +1 """
class Organization(SoftDeleteModel):
    name = models.CharField(max_length=25, null=False, unique=True)
    created_date = models.DateTimeField(auto_now_add=True)
    key = models.UUIDField(
        default=uuid.uuid4, unique=True, editable=False, db_index=True
    )
    api_key = models.CharField(max_length=128, null=True)

    objects = OrganizationManager()

    """ KimWonjong123 """
    class Meta:
        db_table = "organization"
```

Organization 모델의 소스코드

구현 과정 - 백엔드

Pagination

- pagination은 유저 목록을 가져오는 getUsers, 일일 통계를 가져오는 getDailyStats, 검사 결과를 가져오는 getFinalInspecData에 구현 됨
- 해당 api는 두 가지 방식의 pagination을 사용해 구현

오프셋 기반 pagination - getUsers,

getDailyStats

- getUsers, getDailyStats api는 오프셋 기반의 pagination을 사용
- 쿼리 파라미터의 pageSize, pageNumber값을 사용해 쿼리셋을 페이징
- pageSize는 한 페이지의 크기, pageNumber는 데이터를 가져올 페이지의 번호를 의미.

```
queryset = queryset.filter(organization=request.organization)
tot_cnt = queryset.count()
tot_pages = math.ceil(tot_cnt / int(page_size))
users = queryset[
    int(page_size)
    * (int(page_number) - 1) : int(page_size)
    * int(page_number)
]
```

getUsers의 페이징 파트

```
# queryset pagination
n_of_inspec = queryset.values("inspector_id").distinct().count()
stats = queryset[
    int(page_size)
    * (int(page_number) - 1)
    * n_of_inspec : int(page_size)
    * (int(page_number))
    * n_of_inspec
]
tot_cnt = queryset.count() # 전체 데이터 개수
tot_pages = math.ceil(tot_cnt / int(page_size)) # 전체 페이지 개수
if n_of_inspec != 0:
    tot_pages = math.ceil(tot_pages / n_of_inspec)
serializer = self.get_serializer(*args: stats, many=True)
```

getDailyStats의 페이징 파트

구현 과정 - 백엔드

Pagination

- pagination은 유저 목록을 가져오는 getUsers, 일일 통계를 가져오는 getDailyStats, 검사 결과를 가져오는 getFinalInspecData에 구현 됨
- 해당 api는 두 가지 방식의 pagination을 사용해 구현

커서 기반 pagination -

getFinalInspecData

- getFinalInspecData api는 커서 기반의 pagination을 사용
- 쿼리 파라미터의 before, after값과 pageSize를 이용해 쿼리셋을 페이징
- before, after는 요청하는 페이지가 현재 페이지의 다음 페이지인지, 이전 페이지인지 여부에 따라 달라지며 페이징의 커서가 될 데이터의 id값이 전달됨
- 쿼리셋을 페이징 한 후 현재 페이지가 마지막 혹은 첫번째 페이지임을 나타내는 is_first, is_last 값을 업데이트 한 후 response를 보냄

Ex) 현재 페이지의 마지막 데이터 id = 20, 다음 페이지 요청, 페이지 사이즈 = 10

request의 query parameter : after=20&pageSize=10&...

만약 해당 페이지가 마지막 페이지일 경우 response body:

```
{
    isFirst: false,
    isLast: true,
    ...
}
```

```
if cursor is not None:
    if before is not None:
        filtered = queryset.filter(pk__lt=cursor.pk)
        data = list(filtered.reverse()[int(page_size):])
        data.reverse()
        if not filtered[int(page_size):].exists():
            is_first = True
    else:
        queryset = queryset.filter(pk__gt=cursor.pk)
        data = queryset[:int(page_size)]
        if not queryset[int(page_size):].exists():
            is_last = True
else:
    data = queryset[:int(page_size)]
    is_first = True
    if not queryset[int(page_size):].exists():
        is_last = True
```

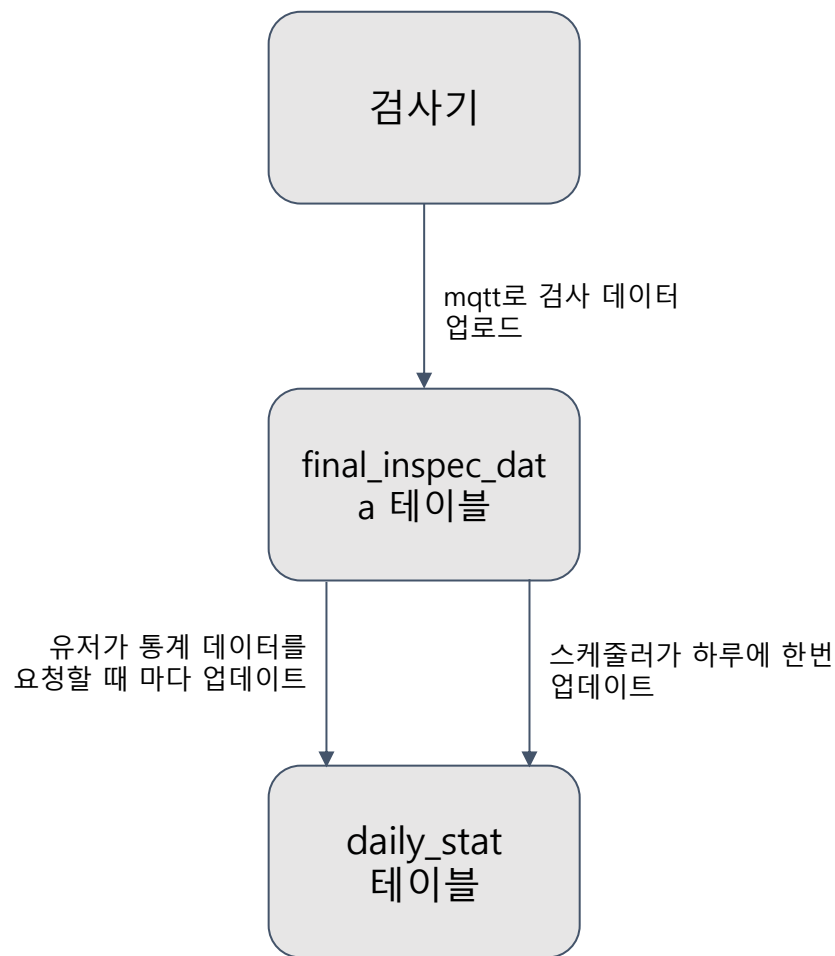
getFinalInspecData의 페이징
파트

구현 과정 - 백엔드

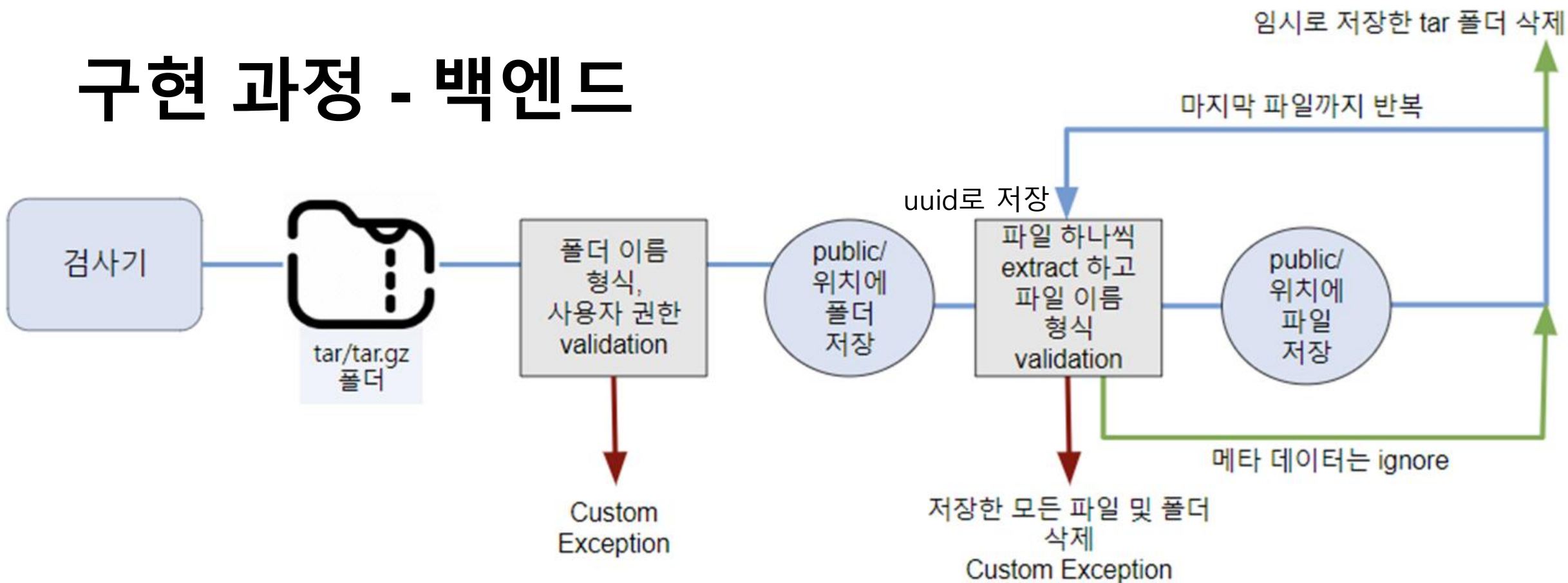
통계 데이터 관리 및 스케줄러

검사가기가 mqtt로 전송한 검사 결과는 DB의 final_inspec_data 테이블에 개별적으로 저장됨

- 검사기의 당일 검사 결과 통계는 DB의 daily_stat 테이블에 검사기별로, 일별로 저장됨
- 사용자가 검사 통계 데이터를 요청할 때 마다 daily_stat 테이블은 자동으로 업데이트 됨.
- 또한 매일 자정에 하루치 검사 결과를 가져와 일일 통계를 갱신하는 스케줄러가 등록되어 작동함.
- 해당 스케줄러는 backend 패키지의 scheduler.py 파일에 구현 됨.
- 누적 통계는 별도로 DB에 저장하지 않고 사용자가 요청할 때 마다 daily_stat 테이블의 값을 가져와 계산함



구현 과정 - 백엔드



파일 이름 중복 방지 및 암호화

- 파일 이름으로 url 접근이 가능하여 원래 파일 이름으로 하였을 시 이름이 중복되거나 보안이 취약해질 수 있음
- 이 점을 고려해 저장하는 모든 파일이 uuid를 파일명으로 가지도록 구현함

tar 및 tar.gz 파일 업로드

tar 파일 형성 시 메타데이터 파일이 만들어짐

- 업로드한 파일 내 사진 저장에 어려움 발생.
- Tar 및 tar.gz 폴더를 압축 해제하여 특정이름으로 시작하는 파일(.) 이나 주어진 확장자 외 파일은 삭제하도록 설정함

MQTT

① message handling (검사기에서 발송되는 메시지를 서버가 처리하는 기능; 검사기 -> 서버)

경로: backend/mqtt.py

- 검사기 여러 대가 하나의 서버 주소에 메시지 발송 ex) intelliz/Frism/settings/

1) Docker 실행 시 서버의 모든 MQTT 주소 구독

```
mqtt_client.subscribe(self.MQTT_PREFIX_SETTINGS)
mqtt_client.subscribe(self.MQTT_PREFIX_INSPEC_RESULT)
mqtt_client.subscribe(self.MQTT_PREFIX_INSPEC_STATE)
```

2) 구독한 주소에서 메시지 발송 시 주소에 따른 handle function 실행

```
payload = json.loads(msg.payload)
if topic == self.MQTT_PREFIX_INSPEC_RESULT:
    self.handle_inspection_result(payload)
if topic == self.MQTT_PREFIX_SETTINGS:
    self.handle_system_setting(payload)
if topic == self.MQTT_PREFIX_INSPEC_STATE:
    self.handle_inspector_state(payload)
```

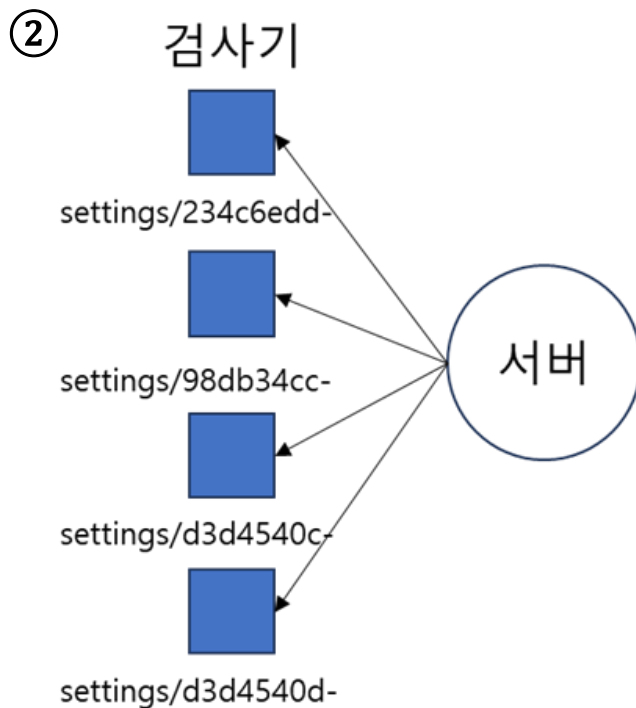
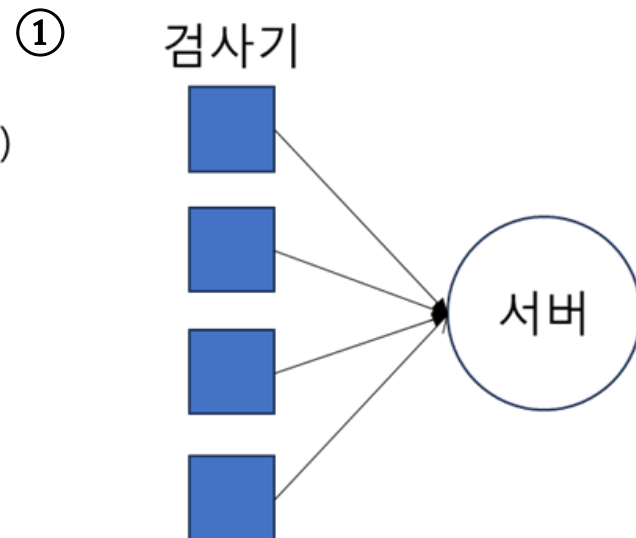
② publish message (서버에서 검사기에 메시지를 발송하는 기능; 서버 -> 검사기)

Ctrl+shift+f "publish.single()"

- 서버가 특정 검사기 한 대를 나타내는 주소에 메시지 발송
- 검사기의 ID인 UUID를 주소에 사용

ex) intelliz/Frism/settings/234c6edd-7bd4-4880-8fa3-2ca28c95c246

- UUID는 dash('-') 포함 필요



MQTT

검사기



서버

검사기 설정 업로드 메시지 감지 주소: MQTT_TOPIC + settings/ 메시지 handling 함수: handle_system_setting()

- 검사기에서 변경된 설정값을 서버에 업로드하기 위한 MQTT 메시지를 처리하는 함수
 - 대시보드 '설정' 탭에서 바꿀 수 있는 모든 변수 업데이트
 - Inspector, CCTV, Recipe, LED 및 LED_Channel, Camera 및 DNN_Class 모델의 elements를 업데이트

```
{
  "uid": "234c6edd-7bd4-4880-8fa3-2ca28c95c246", // 변경된 inspector id
  "recipes": {
    "three": { // Recipe의 column "three" 는 recipe_sub_info
      "inspectors": [ // Inspector의 column
        {
          "inspectorID": 0,
          "inspectorSubInfo": "none",
          "cameraInfo": { // Camera의 column
            "cameraSetUpSubInfo": "none",
            "triggerMode": "Software", //trigger_info
            "cameraId": "Basler a2A1920-165g5m (40362583)",
            "exTimeVal": 857,
            "digitalShiftVal": 2,
            "gammaVal": 1,
            "gainVal": 0,
            "limitClaheVal": 5,
            "tileSizeClaheVal": 5,
            "imageWidth": 1920,
            "imageHeight": 1200
          },
          "dnnSetUp": { // DNN_class의 column
            "dnnSetUpSubInfo": "none",
            "dnnPath": "D:\\workspace\\frism_inspection_r",
            "gpuID": 0, //gpu_num
            "dnnClassInfo": [
              {
                "className": "background", //name
                "classID": 0, //sub_id
                "classPVal": 0.5, //p_val
                "DefectMinSize": 5
              }
            ]
          }
        }
      ]
    }
  }
}
```

```
"basicInfo": { // LED의 column
  "ioDelayTime": 0,
  "ledInfos": [
    {
      "ledID": "0_test",
      "channels": [ // LED_channel의 column
        {
          "channelName": "0",
          "channelNumber": 0,
          "channelValue": 168
        }
      ]
    }
  ]
},
"cctv": { // CCTV의 column
  "cctvSubInfo": "none", //video_url
  "cctvId": 0
},
"systemInfo": { // inspector의 column
  "ioFilePath": "C:\\Advantech\\DAQNavi\\DAQNavi\\NavigatorPlug",
  "imageSaveFolder": "none" //image_path
}
```

i) **Bold체**: 구역의 제목 역할-> 수정 필요 X
ii) 그 외: 각각 recipe, inspector, camera 등의 column 이름으로, camel case로 설정되어 있음 (column 이름과 완전히 똑같지 않은 것은 주석으로 정리해 두었음)

- 여러 모델에 걸쳐 작성된 메시지이기 때문에 관련 모델을 모두 삭제 후 다시 만드는 식으로 구현됨

* 괄호 형식만 맞으면 부분적으로도 작동이 가능함 -> 테스트 시 활용 추천

MQTT

검사기



서버

검사기 설정 업데이트 메시지 전송 주소: MQTT_TOPIC + settings/ + 해당 검사기 UUID

‘검사기 설정 업로드’와 **반대로** 설정 탭에서 변경된 설정값을 검사기에 반영하기 위한 MQTT 메시지를 처리하는 함수

- 대시보드 ‘설정’ 탭에서 바꿀 수 있는 모든 변수 전송
- Inspector, CCTV, Recipe, LED 및 LED_Channel, Camera 및 DNN_Class 모델의 elements를 업데이트
- 관련된 모델의 patch 함수(update_xxx) 실행 시 해당 검사기를 가리키는 MQTT 주소에 전송됨
- Json 메시지 포맷은 ‘검사기 설정 업로드’ 시 사용되는 것과 거의 유사하나, 부가적으로 model 개수가 포함되어 있으며, 검사기 id를 따로 포함할 필요가 없음

```
{  검사기의 id 없음
  "numRecipes": 1, // 레시피 개수를 나타냄
  "recipes": {
    "test1": {
      "recipeSubInfo": "none",
      "numInspectors": 1, // 검사기 개수를 나타냄
      "inspectors": [
        {
          "inspectorID": 0,
          "inspectorSubInfo": "none",
          "cameraInfo": {
            "cameraSetUpSubInfo": "none",
            "cameraId": null,
            "exTimeVal": 0,
            "gainVal": 0,
            "imageWidth": 0,
            "imageHeight": 0
          }
        }
      ]
    }
  },
  "numModels": 1, // 모델 개수를 나타냄
  "models": {
    "test1": {
      "modelSubInfo": "none",
      "numElements": 1, // 요소 개수를 나타냄
      "elements": [
        {
          "elementID": 0,
          "elementSubInfo": "none",
          "elementType": "recipe",
          "elementValue": "test1"
        }
      ]
    }
  }
}
```

레시피, 검사기 등의 개수를 나타내는 메시지가 추가됨

MQTT

검사기



서버

검사기 결과 업로드 메시지 감지 주소: MQTT_TOPIC + inspec_result/ 메시지 handling 함수: handle_inspection_result()

- 검사기에서 도출된 결과를 서버에 업로드하기 위한 MQTT 메시지를 처리하는 함수
 - inspec_data 모델의 elements를 새로 생성하여 저장
 - 대시보드 탭에 나타나는 값들을 전송

```
{
  uid: number,
  name: string, (recipe name)
  finalInspecData: {
    id: string,
    result: string, (OK | NG)
    inferTime: number,
    inspecDate: string, (ISO DateTime Format)
  },
  inspecData: [
    {
      inspectorId: number, (camera id)
      defectSize: number,
      result: string, (OK | NG)
    },
    ...
  ]
}
```

ex)

```
JSON
{
  "uid": 1,
  "name": "two",
  "finalInspecData": {
    "id": "test",
    "result": "NG",
    "inferTime": 0.3,
    "inspecDate": "2023-07-25T13:48:01+09:00"
  },
  "inspecData": [
    {
      "inspectorId": 1,
      "defectSize": 1,
      "result": "NG"
    },
    {
      "inspectorId": 2,
      "defectSize": 0,
      "result": "OK"
    }
  ]
}
```

- i) **Bold체**: 구역의 제목 역할-> 수정 필요 X
- ii) **그 외**: 각각 final_inspec_data 혹은 inspec_data의 column 이름으로, camel case로 설정되어 있음 (오른쪽 사진은 예시)

MQTT

검사기



서버

검사기 PC 상태 업로드 메시지 감지 주소: MQTT_TOPIC + inspec_state/ 메시지 handling 함수: handle_inspection_state()

- 검사기의 현재 PC 상태를 서버에 업로드하기 위한 MQTT 메시지를 처리하는 함수
 - Inspector_state 모델의 elements를 새로 생성하여 저장 (수정하는 방식에서 변경)
 - 검사기 용량과 온도에 대한 데이터

```
if inspector_state.ratio >= float(percentage):  
    message = {"message": "storage alert"}  
    publish.single(  
        self.MQTT_PREFIX_INSPEC_STATE + str(inspector.id),  
        json.dumps(camelize(message), indent=4),  
        hostname="broker.hivemq.com",  
        port=1883,  
    )
```

```
if (  
    inspector_state.ratio >= float(percentage)  
    and inspector_state.email_sent == 0  
):  
    with transaction.atomic():  
        inspector_state.email_sent = 1  
        inspector_state.save()
```

```
if inspector_state.ratio > float(percentage):  
    with transaction.atomic():  
        inspector_state.email_sent = 0  
        inspector_state.save()
```

용량 부족의 기준

사용 용량 / (검사기 남은 용량 + 사용한 용량)이
설정된 STORAGE_ALERT_PERCENTAGE 보다 높을 경우

용량 부족 알림의 방식

1) MQTT로 메시지 전송

메시지 발송 주소: MQTT_TOPIC + inspec_state/ + 해당 검사기 UUID
검사기

2) Email로 메시지 전송

email_sent 변수(초기값=0)를 1로 설정 후 이메일 발송

→ 이후 scheduler에서 email_sent가 1일 시 이메일 주기마다 발송

검사기



서버

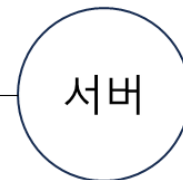
용량 개선 확인 시 email_sent를 다시 0으로 설정

MQTT

검사기



서버



검사기 결과값 이미지 요청 메시지 전송 주소: MQTT_TOPIC + image/ + 해당 검사기 UUID

- 검사기 결과값에 대한 이미지는 upload_file api를 통해 등록되며, 이 때 image_url 혹은 UUID도 함께 저장됨

```
inspec_data.image_id = file_data.uuid
inspec_data.save()
```

```
for d in data.get("contents"):
    if d.get("img_url"):
        d[
            "img_url"
        ] = f"{settings.DOMAIN_BACKEND}/{settings.API_VERSION}/files/"
    else:
        inspector_id = final_inspec_data.inspector_id
        message = {
            "final_inspec_data_sub_id": final_inspec_data.sub_id
        }
        publish.single(
            settings.MQTT_TOPIC
            + settings.MQTT_PREFIX_IMAGE
            + str(inspector_id),
            json.dumps(*args: message, indent=4),
            hostname="broker.hivemq.com",
            port=1883,
        )
```

검사기 결과값 조회 시

- 1) 이미지 주소가 저장되어 있을 시
-> 해당 이미지 주소에서 이미지를 불러오는 것을 성공
- 2) 이미지 주소가 저장되어 있지 않을 시
-> 해당 검사기 결과값의 최종값(final_inspec_data)의 sub id를 검사기에 전송하여 이미지 등록 필요 알림을 전송

용량 부족 이메일 발송 기능



이메일 발송 대상

해당 검사기를 보유한 고객사의
모든 CUSTOMER_SUPER

```
users = User.objects.filter(
    organization=inspector_state.inspector.organization.id,
    permission="CUSTOMER_SUPER",
)
```

이메일 발송 주기

STORAGE_ALERT_PERIOD로 변경
가능; '일' 단위로 전송

```
days=int(env("STORAGE_ALERT_PERIOD"))
```

* days -> hours, minutes, seconds 등의 단위로 변경 가능
'일'이 용량 개선에 가장 적합한 단위라고 판단하여 사용함

이메일 형식 (디자인/내용)

storage_alert_content.html과
storage_alert_cotent.txt 에서 제어

* 메일의 버튼 클릭 시 intelliz 본사 사이트로 이동하도록 설정됨

작동 방식

python manage.py command 명령어 감지
시 백그라운드에서 스케줄러를 시작하여
작동하며, 이는 docker-compose up --build
명령어로 같이 실행되도록 설정함

- 감지 명령어 설정: execute_from_command_line() 함수 사용
- Docker 명령어와 함께 실행되도록 제어: docker-compose.yml에서 제어

반복 업무 추가 방법

업무 수행 함수를 command.py에 선언 후
handler() 함수 내에서
scheduler.add_job(함수, interval, 주기) 작성

```
scheduler = BackgroundScheduler()
scheduler.add_job(send_storage_email_alert, trigger='interval', days=i
scheduler.start()
```

MQTT와 이메일 발송 기능 개발 시 발생 문제와 해결 과정

사용자를 고려하지 못한 구현 방법 1 – 이메일 발송 주기

개발 초기에는 주기마다 용량 부족 이메일을 발송하지 않고, **단 한번만** 발송하는 식으로 구현하였음

- 실제 검사기 관리자는 한번의 이메일로 용량 개선이 즉각적으로 이루어지지 않을 수 있음; 인지하더라도 지속적인 리마인드 필요
- 여러 번 알림을 발송하는 필요성을 느낌 -> apscheduler로 이메일을 주기마다 발송하도록 변경

사용자를 고려하지 못한 구현 방법 2 – 전송 메일 내용

메일 내용 내 **검사기 id**만으로 무슨 검사기의 PC 용량이 부족한지 한눈에 파악하기 어려움

- 보안을 위해 uuid로 검사기 id가 변경되며 사용자가 무슨 검사기가 용량 개선이 필요한지 알기 더 어려워짐
- 사용자가 검사기 메모를 기록하는 inspector 모델의 '**memo**' element를 발송하는 메일에 id와 함께 추가함

검사기 PC 상태 기록

기존에는 하나의 검사기 PC 상태를 계속해서 수정하는 식으로 구현

- 하나의 검사기 상태가 수정하는 식으로 구현되어 이전의 검사기 상태 데이터가 남지 않음
- 이전 검사기 상태 역시 기록하기 위해 검사기 상태가 MQTT 발송 시마다 새로 생성되도록 변경; 검사기 상태를 불러오는 api도 여러 개를 동시에 불러올 수 있는 형태로 변경

MQTT 메시지 serializer 사용

설정 변경 시 전송하는 MQTT의 길이가 복잡하고 길

- API 뿐만 아니라 MQTT 메시지도 serializer로 쉽게 만들 수 있도록 구현
- Serializer를 사용함으로써 DB에서 json 형식으로 자동으로 변환되도록 구현

구현 과정 - 프론트엔드

프로젝트 구조

- 대부분의 관련 파일은 /src 내부에 있음
- 각각의 기능에 따라 /src 아래에 폴더로 배치

components

- 새로 정의한 리액트 컴포넌트
- common - 모든 페이지에서 공통으로 사용되는 컴포넌트
- pages - 페이지 별로 사용되는 컴포넌트
- sections - 각 페이지 최상위에서 섹션을 나누기 위해 사용되는 컴포넌트
- (페이지 최상위에는 가능한한 섹션 컴포넌트만 존재하도록 구성함)

constants

- 글로벌하게 사용되는 상수
- api - api 관련 exception, api 요청 url 등을 포함

contexts

- 프로젝트 최상위에 존재하는 새로 정의한 글로벌 컨텍스트

hooks

- 새로 정의한 react hook

mockups

- 개발 과정 또는 데모 유저를 위한 mock 데이터

pages

- 페이지 컴포넌트
- File Based Routing의 기준점

types

- 새로 정의한 type 또는 interface
- api 응답 형식 포함

utils

- 새로 정의한 유틸함수류
- 반복 연산 및 api fetcher 등 포함

```
public
└─ src
  └─ assets
  └─ components
    └─ common
      └─ AppShell
      └─ CenterWrapper
      └─ CenteredLoader
      └─ InfoRow
      └─ Modal
      └─ PageTitleWrapper
      └─ SubAndList
      └─ TextAndCopy
    └─ pages
      └─ dashboard
        └─ InspecDataSlide
        └─ ResponsiveList
      └─ inspectors
      └─ manage
        └─ DesktopUserList
        └─ MobileUserList
        └─ OrgList
      └─ settings
    └─ sections
      └─ account
        └─ info
        └─ login
        └─ password-reset
        └─ register
      └─ cctv
      └─ common
        └─ SelectorSection
      └─ dashboard
      └─ inspectors
        └─ $key
      └─ manage
        └─ admin
        └─ staff
  └─ constants
    └─ api
  └─ contexts
    └─ Auth
    └─ ViewType
  └─ hooks
  └─ mockups
  └─ pages
    └─ account
      └─ password-reset
    └─ inspectors
    └─ manage
      └─ admin
      └─ organizations
  └─ types
    └─ ListElement
    └─ api
  └─ utils
    └─ notifications
```

구현 과정 - 프론트엔드

File Based Routing

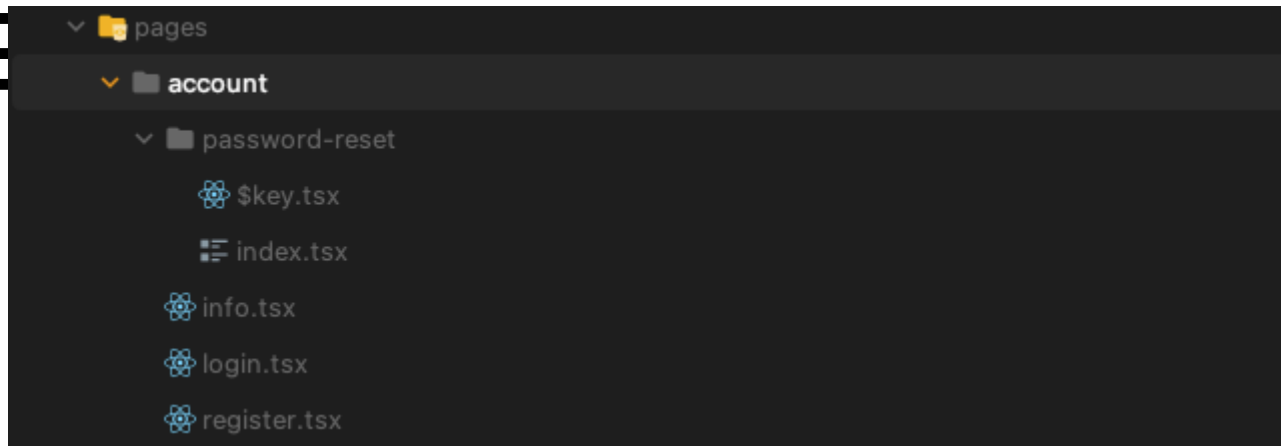
- 페이지 추가시 특정 폴더의 내부 구조를 기반으로 페이지를 생성함
- 매번 react-router-dom의 Route 컴포넌트를 추가할 필요가 없음

Static Routing

- index.tsx 파일의 페이지 컴포넌트는 해당 폴더의 root 경로 페이지로 추가됨
- {index를 제외한 이름}.tsx 파일의 페이지 컴포넌트는 해당 경로에 맞게 추가 됨

Dynamic Routing

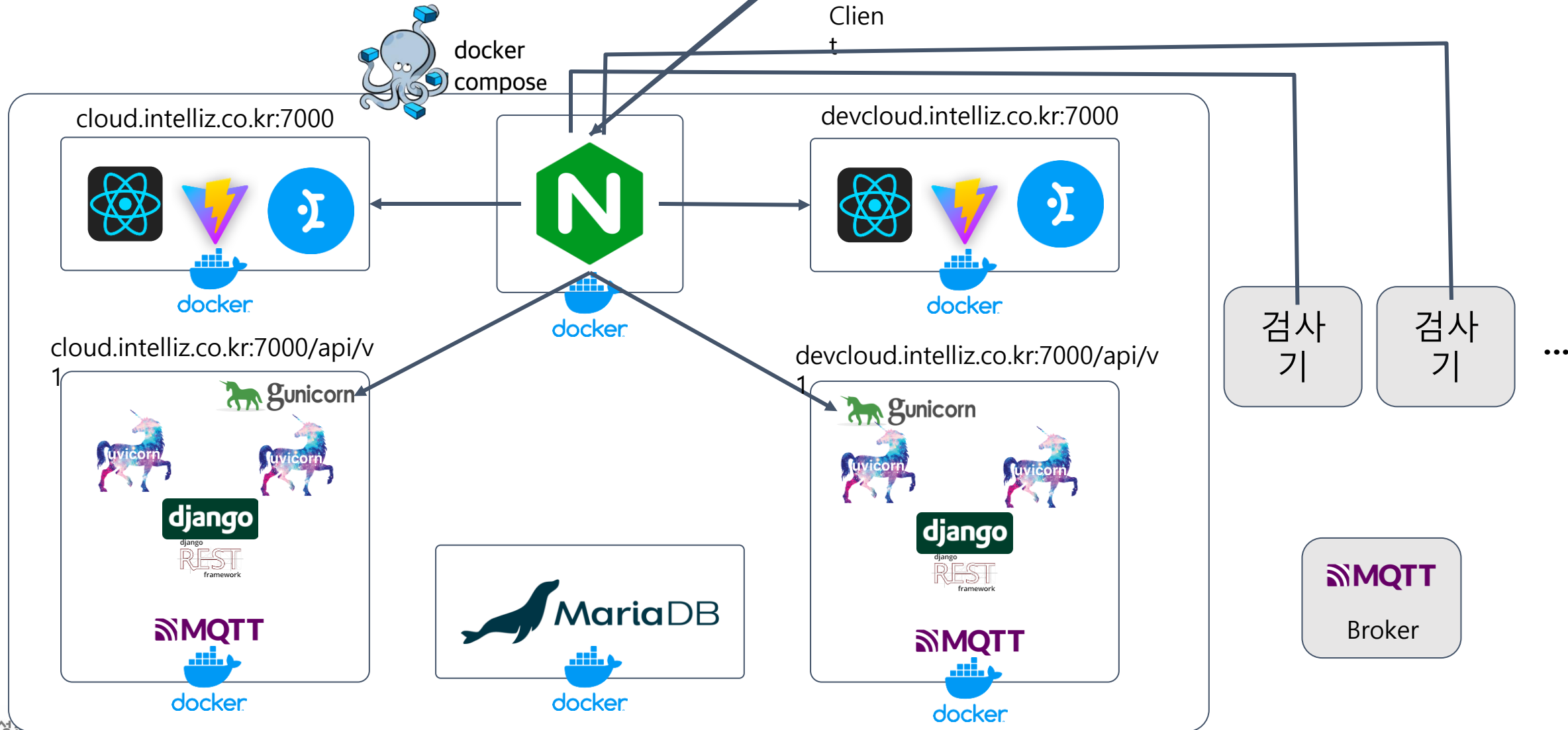
- \${파라미터 명}.tsx 파일, 또는 폴더는 해당 이름의 route 파라미터에 대응 됨
- 각각의 파라미터는 기존의 React와 같이 useParams() hook을 통해 사용 가능함



```
28  
29 const { key : string | undefined } = useParams();  
30 const navigate : NavigateFunction = useNavigate();
```

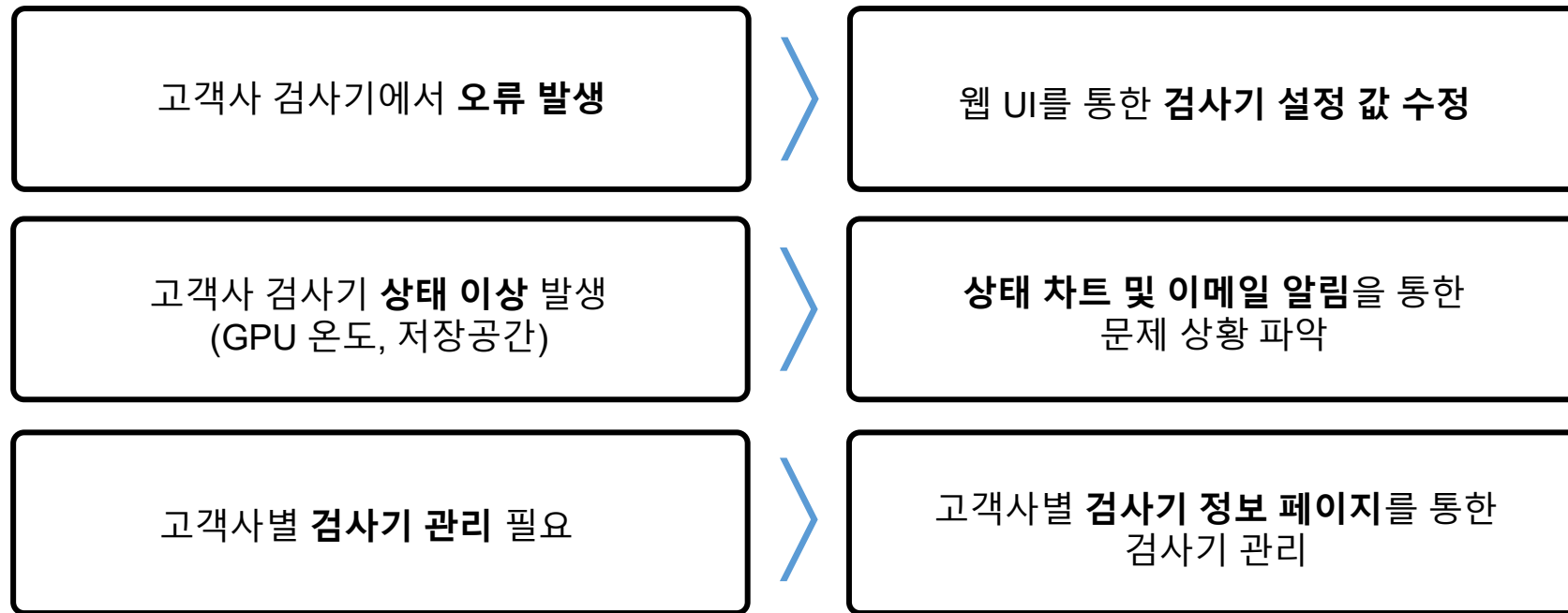
File Based Routing 및 useParams() hook 예시

구현 과정 - 서비스 아키텍처



적용 기술 - 주요 시나리오

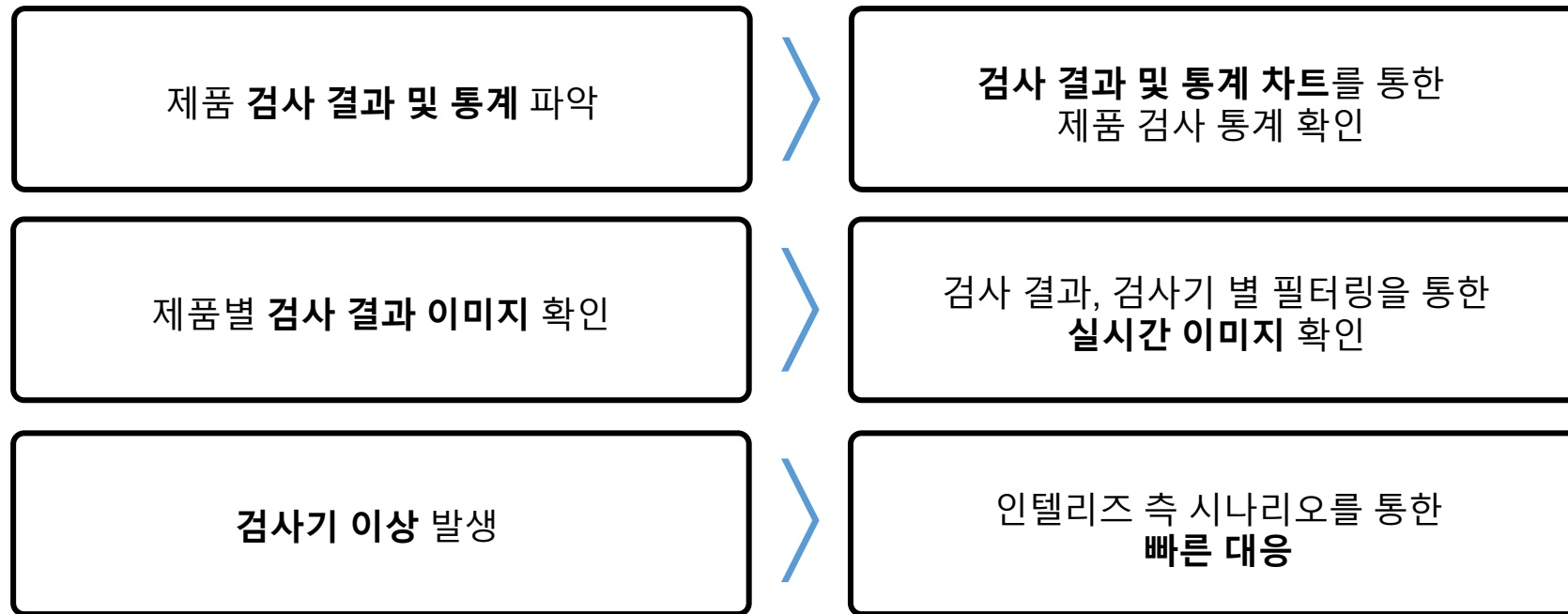
인텔리즈



⇒ 고객사 현장 방문 최소화 및
즉각적인 상태 파악

적용 기술 - 주요 시나리오

고객사



⇒ 검사 결과 실시간 확인 및
검사기 이상에 대한 빠른 지원

기술의 차별성 및 독창성

검사기 관리 측면

검사기 설정 값으로 인한 이상 발생시
웹 UI를 통해 실시간으로 수정 가능

⇒ 고객사에 빠른 사후 지원 제공

검사기 저장공간 초과 등 문제 발생시
이메일을 통한 실시간 알림 가능

⇒ 문제 상황 발생시 빠른 인지 및 대응

검사 결과 확인 측면

검사 결과 통계 및 실시간 검사 이미지
확인 가능

⇒ 단순 결과 확인,
검사기 설정 값 적절성 파악

과제 성과 및 기대효과

대시보드를 통한 원격 관리
현장을 직접 방문하지 않고
원격으로 상태를 손쉽게 관리할 수 있어
관리 편의성 증가

모니터링 및 알림 발송
CCTV를 통한 검사기 실시간 확인 및
저장공간 부족, 과열 등의 이상 발생 시
자동 알림 발송을 통한 검사기 유지 보수

데이터 시각화
검사 결과 이미지를 확인하고
시각화된 검사 결과 통계 데이터를
웹상으로 간편하게 확인

빠른 지원을 통한
서비스 품질 향상

검사기 유지 보수 및
관리 편의성 증대

데이터 시각화를 통한
차별화된 서비스

개선점 및 향후 계획

실시간 로거 연결

현재 백엔드는 파이썬 내장 로거를 사용
실시간으로 올라오는 에러 메시지를 읽기 부적합
적절한 실시간 로거 연결

상용 배포 세팅

상용 배포에 적합한 dockerfile 작성
및
백엔드 설정 확인

이미지 표현 방식 변경

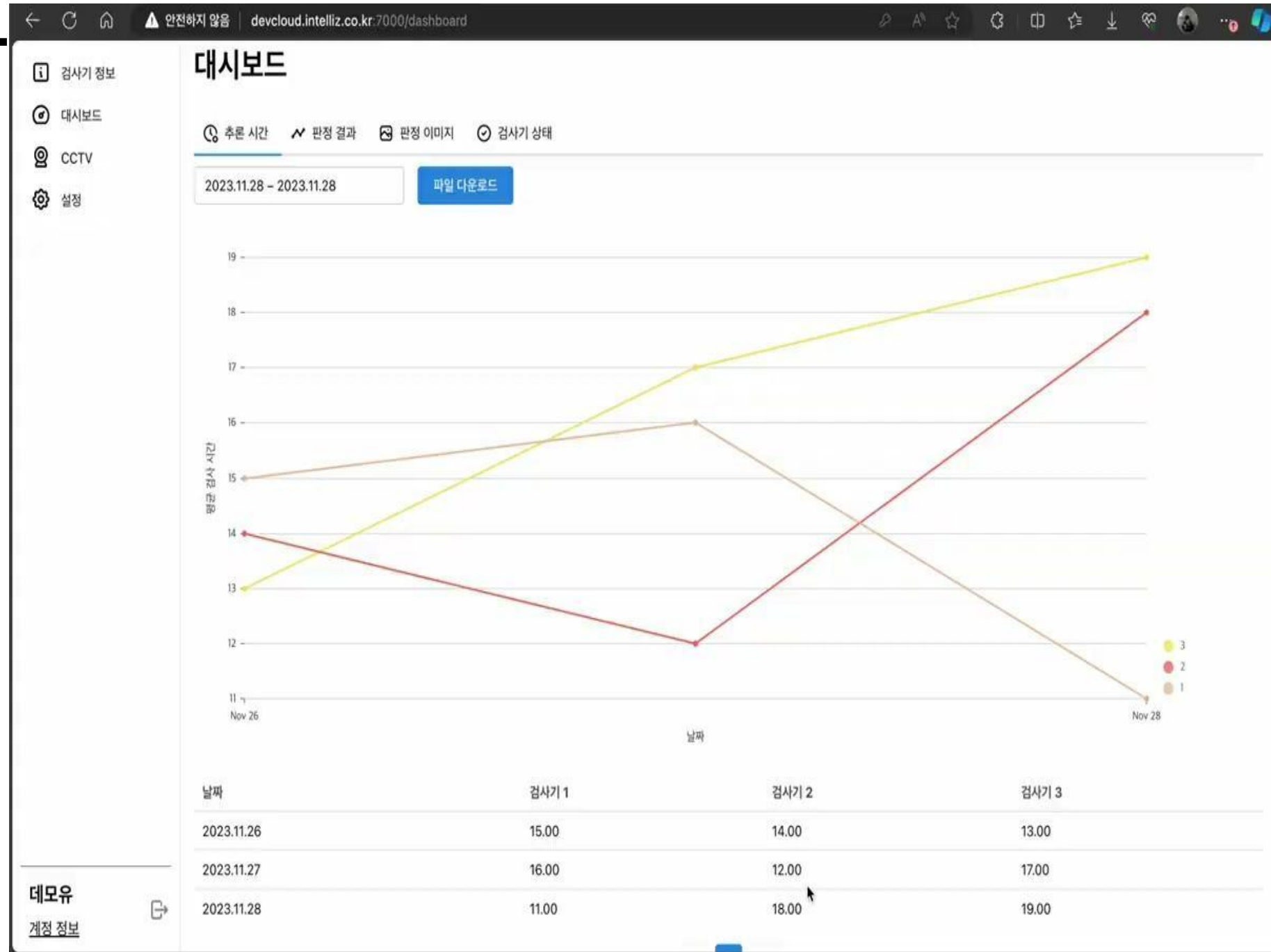
검사 결과 테이블에 이미지 contour 데이터 추가 및 react-conva를 이용한 UI 표현 방식 변경

통합 테스트

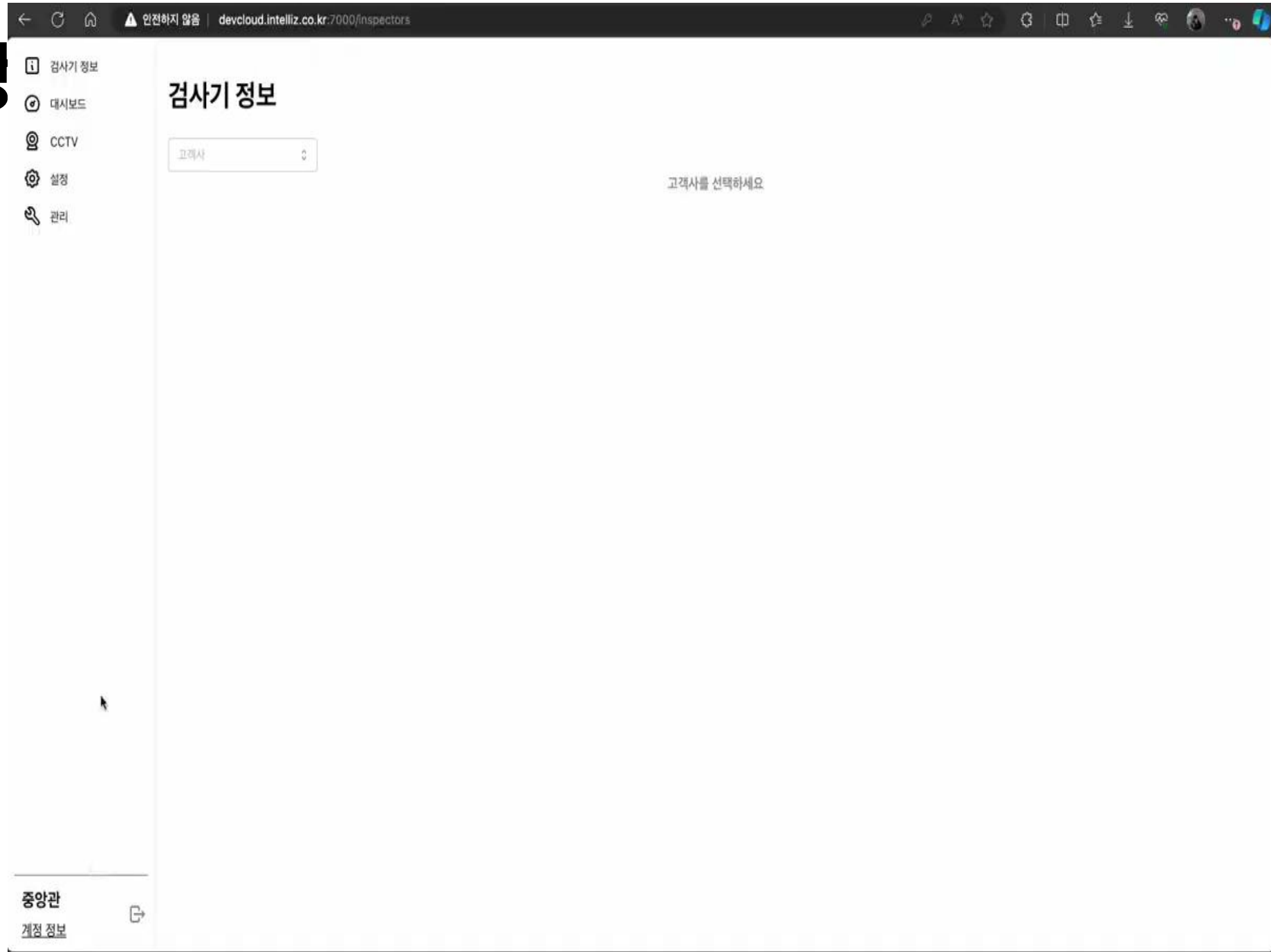
테스트 전용 검사기 추가 후
검사 데이터, 설정 값 등 업로드
테스트

시연 동영상

시연 동영상



시연 동영상



Appendix

- 2023년 산학협력과제 수행 소감
- 2023년 산학협력 과제 요약

2023년 산학협력과제 수행 소감

□ 이강훈

- 팀장으로 근무를 수행하면서 업무 분담 등 팀 프로젝트에서 고려해야 할 사항들이 무엇인지 생각해볼 수 있는 기회가 되었습니다. 하계집중근무와 이후에 세부적인 내용들을 완성하고 과제를 마무리하면서, 담당자가 변경되는 인수인계 과정에서의 문서화의 중요성들을 많이 알게 되었습니다.

□ 김원종

- 실무 프로젝트를 진행하면서 스스로 공부하는 능력을 많이 기를 수 있었습니다. 또한 다양한 프레임워크와 라이브러리를 사용하면서 웹 개발에 대한 전반적인 지식을 쌓을 수 있었고 협업 능력또한 기를 수 있었습니다.

□ 김채리

- 실무를 통해 Django와 Git을 본격적으로 사용하고 관련 개발 능력을 키울 수 있었습니다. 더 나아가 스스로 효율적으로 학습하고 개발하는 방법을 깨닫고, 개발 내용을 타인에게 자세하게 발표하는 경험을 할 수 있었습니다. 학교 전공 수업 외 진로 관련 활동이 적었는데, 산학협력프로젝트가 열정적인 진로 탐색의 시발점이 된 것 같습니다.

2023 산학협력 과제 요약

과제명	클라우드 기반의 로봇 및 검사기 모니터링 시스템	기업명 / 담당자	인텔리즈 / 나현식
과제개요	- 원격으로 검사기 상태를 확인 및 제어하는 플랫폼 필요 - MQTT, HTTP 통신을 통한 엣지디바이스, 검사기 연동 - 통합 설비 관리로 고객 피드백 및 딥러닝 성능 개선에 기여	참여 학생	이강훈 (소프트웨어학과, 2학년)
			김원종 (소프트웨어학과, 3학년)
			김채리 (소프트웨어학과, 2학년)
과제 기간	2023년 4월 10일 ~ 2023년 12월 31일	지도교수/멘토	이주식 교수 / 김선민 연구원, 이원용 연구원
		산학교수	이주식 교수
결과물	- 검사기 엣지 디바이스 관리 시스템 - 검사 결과, 검사기 설정값, 검사기 상태 관리 - 사용자 및 고객사 관리		